A REPORT OF INDUSTRIAL TRAINING

at

BABU BANARASI DAS UNIVERSITY

LUCKNOW

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD

OF THE DEGREE OF

BACHELOR OF TECHNOLOGY

(Computer Science and Engineering)



BBD UNIVERSITY

2026–27

PROJECT TITLE

FLIXRECOMMEND: HYBRID MOVIE RECOMMENDATION PLATFORM

SUBMITTED BY:

Mihir Dutta

UNIVERSITY ROLL NO.: 1230438030

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BABU BANARASI DAS UNIVERSITY,

LUCKNOW

CERTIFICATE BY COMPANY / INDUSTRY / INSTITUTE

This page is reserved for the authentic industrial training certificate issued by the company, industry or institute. No certificate, signature, seal, training organisation or training dates have been created for this report.

Attach the original, verified certificate here before final submission and binding.

Training particulars to verify

Particular	Information
Training organisation	**Grastech**
Training period	From **22 June 2026** to **05 Aug 2026**
Certificate / reference number	**GRAS/PBT/ML/111**
Industry supervisor	**Mr.Himesh Singh**

BABU BANARASI DAS UNIVERSITY, LUCKNOW

CANDIDATE'S DECLARATION

I, Diya Verma, hereby declare that this report entitled “FlixRecommend: Hybrid Movie Recommendation Platform” documents the project and technical work described in the project README files and the attached screenshots. It is submitted in partial fulfilment of the requirements for the degree of Bachelor of Technology (Computer Science and Engineering) at Babu Banarasi Das University, Lucknow.

The training organisation and training period were not provided in the supplied materials and must be completed and verified before submission. I confirm that the project descriptions in this report are based on the supplied README files and images; the attached screenshots are presented as project evidence, not as independent verification of a live deployment.

Name of Student: Mihir Dutta

University Roll No.: 1230438030

Signature of Student: _____

Date: _____

For Official Use Only

The Industrial Training Viva–Voce Examination of _____ was held on _____ and accepted.

Signature of Internal Examiner: _____

ABSTRACT

FlixRecommend is a movie-discovery application that combines a React-based frontend with a FastAPI recommendation service. The supplied project documentation describes a hybrid recommender that uses content similarity and collaborative filtering to rank movies for a signed-in viewer. The content signal is based on TF–IDF representations of movie genres and cosine similarity; the collaborative signal is generated by a trained Singular Value Decomposition (SVD) model. The backend normalises the two score sets and blends them using a configurable content weight.

The frontend provides account access, first-time taste onboarding, movie discovery, search and genre filters, sorting, a saved-movie profile, movie details and playback. Authenticated likes and watch events are sent to the backend; a watch event at or above 50% can add a movie to the user's taste profile. The backend is documented as a FastAPI service using SQLAlchemy and PostgreSQL, JWT authentication, and generated API documentation.

This report follows the chapter order and academic presentation requirements in the supplied BBDU ITE format and uses the Food Recipe report as a structural reference. All five project screenshots supplied with the request are included. The README also states that three trained data/model artifacts are absent from the provided working tree; therefore, actual model execution, test outcomes, training-provider details and training dates are not claimed as independently verified.

Keywords: Movie recommendation, TF–IDF, cosine similarity, collaborative filtering, SVD, FastAPI, React, PostgreSQL, JWT.

ACKNOWLEDGEMENT

I express my sincere gratitude to Babu Banarasi Das University and the Department of Computer Science and Engineering for providing the academic setting in which project-based learning can be pursued.

I am thankful to the faculty members, mentors and peers whose guidance and feedback support the development of technical skills. I also acknowledge the maintainers of the technologies and documentation referenced in this report, including React, FastAPI, SQLAlchemy, scikit-learn, scikit-surprise and PostgreSQL.

Finally, I acknowledge the project documentation and screenshots supplied for FlixRecommend. They provide the factual basis for the system description and visual evidence included in this report.

ABOUT THE COMPANY / INDUSTRY

The supplied README files describe FlixRecommend as an educational movie discovery and recommendation project. They do not identify a training company, industry partner, supervisor or training venue. These details are therefore intentionally left for verification rather than inferred.

Project context

FlixRecommend comprises a browser-based movie catalogue and a separate FastAPI recommendation backend. Its design combines a content-based score with a collaborative score to generate personalised rankings. The frontend is described as React with Vite and Tailwind CSS; the service uses Python, FastAPI, SQLAlchemy and PostgreSQL. The supplied project README notes that the repository includes a local SQLite file that the application does not use.

Training details

Item	Verified information from supplied materials
Training organisation	**Grastech**
Training venue	**BBDU-lab**
Training period	**22 June 2026 to 05 Aug 2026**
Certificate number	**GRAS/PBT/ML/111**
Project reference	FlixRecommend movie-discovery frontend and hybrid recommendation API

LIST OF FIGURES

Figure	Description	Page
Figure 6.1	Sign-in screen for personalised movie discovery	14
Figure 6.2	Featured movie panel and recommendation status	15
Figure 6.3	Movie catalogue with match indicators	16
Figure 6.4	Notebook cell for TF–IDF and cosine similarity	17
Figure 6.5	Notebook cell for SVD collaborative model training	18

LIST OF TABLES

Table	Description	Chapter
Table 1.1	Software and hardware requirements	1
Table 1.2	Technology stack and purpose	1
Table 2.1	High-level system architecture	2
Table 2.2	Recommendation signals and blending	2
Table 2.3	Major application modules	2
Table 3.1	Typical viewer workflow	3
Table 3.2	Functional checks and evidence status	3
Table 6.1	Supplied project screenshots	6

DEFINITIONS, ACRONYMS AND ABBREVIATIONS

Term	Meaning
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
CORS	Cross-Origin Resource Sharing
JWT	JSON Web Token
ML	Machine Learning
ORM	Object-Relational Mapping
REST	Representational State Transfer
SVD	Singular Value Decomposition
TF–IDF	Term Frequency–Inverse Document Frequency
UI	User Interface
UX	User Experience
Vite	Frontend build and development tool

CONTENTS

Topic	Page No.
Certificate by Company / Industry / Institute	i
Candidate's Declaration	ii
Abstract	iii
Acknowledgement	iv
About the Company / Industry	v
List of Figures	vi
List of Tables	vii
Definitions, Acronyms and Abbreviations	viii
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope	2
1.5 Software and Hardware Requirements	3
1.6 Technology Stack	3
CHAPTER 2: TRAINING WORK UNDERTAKEN	4
2.1 Development Methodology	4
2.2 Requirement Analysis	4
2.3 System Architecture	5
2.4 Data and Model Preparation	6
2.5 Authentication and API Integration	7
2.6 Feature Implementation	7
2.7 Testing and Deployment Workflow	8
CHAPTER 3: RESULTS AND DISCUSSION / OBSERVATION	9
3.1 Implemented Application Modules	9
3.2 User Workflow	10
3.3 Recommendation Method	10
3.4 Functional Testing and Evidence	11
3.5 Observations	12
3.6 Limitations	12
CHAPTER 4: CONCLUSION	13
CHAPTER 5: REFERENCES	14
CHAPTER 6: APPENDIX	15

CHAPTER 1: INTRODUCTION

1.1 Background

Digital entertainment services offer large film catalogues, which can make it difficult for viewers to identify titles that fit their interests. A recommendation system addresses this discovery problem by ranking items using information about their content and signals from user preferences. FlixRecommend is documented as a movie-discovery frontend paired with a backend service that produces hybrid, personalised recommendations.

1.2 Problem Statement

A catalogue-only browsing experience requires viewers to search and compare titles manually. A useful personalised feed must reflect the characteristics of movies a viewer likes while also using learned patterns from rating data. The project therefore combines a content-based signal with a collaborative signal and exposes the resulting ranked list through an authenticated API.

1.3 Objectives

– Build a movie-discovery interface with search, genre filters and sorting.

– Support user signup and login, taste onboarding and saved movie preferences.

– Generate recommendations through a backend hybrid-ranking endpoint.

– Combine TF–IDF content similarity with SVD-based collaborative predictions.

– Provide movie details, related titles and playback where a stream is available.

– Keep the recommendation feed in an explicit unavailable state if the backend ranking service fails.

1.4 Scope

The documented frontend supports movie browsing, sign-in and sign-up, initial taste selection, profile and saved-movie views, search, genre filtering, sorting, details, likes and

playback. The backend provides account endpoints, authenticated taste updates, watch-event recording, ranked recommendations and stream lookup. The supplied documentation describes a hybrid score controlled by a configurable content weight.

The report does not treat missing trained artifacts or undocumented training particulars as completed evidence. The README says that movies_data.pkl, similarity_matrix.pkl and svd_collaborative_model.pkl are required beside the backend entry point but are absent from the supplied working tree. Live deployment URLs, performance benchmarks and test-suite results were not supplied.

1.5 Software and Hardware Requirements

Table 1.1: Software and hardware requirements

Category	Requirement / technology
Frontend	Node.js 20.19+ or 22.12+, React 19, Vite 8
Backend	Python 3.13 or a compatible version, FastAPI, Uvicorn
Database	PostgreSQL connection configured through DATABASE_URL
Recommendation	pandas, NumPy, scikit-learn, scikit-surprise, joblib
Authentication	JWT signing secret and bcrypt password hashing
Development	Code editor, browser, Git, API test client
Hardware	Computer with internet connection and modern browser

1.6 Technology Stack

Table 1.2: Technologies used in FlixRecommend

Technology	Purpose
React 19 / Vite 8	Movie-discovery user interface and frontend development server
Tailwind CSS 3	Utility-based frontend styling
Axios / Hls.js	HTTP communication and HLS video playback
Python / FastAPI / Uvicorn	Recommendation and account API service
PostgreSQL / SQLAlchemy	Persistent relational data and ORM access
pandas / NumPy	Movie and recommendation data processing
scikit-learn	TF–IDF representation and cosine-based content signal
scikit-surprise SVD	Collaborative rating prediction
python-jose / bcrypt	JWT authentication and password hashing

Technology	Purpose
joblib	Loading saved recommendation model artifacts

CHAPTER 2: TRAINING WORK UNDERTAKEN

2.1 Development Methodology

The supplied project notes describe a frontend and backend developed as separate applications. A practical workflow for the documented design is incremental: define a viewer-facing feature, implement its UI and API interaction, validate the request and response, then review the result in the browser. Changes to the recommendation pipeline also require the catalogue, similarity matrix and fitted collaborative model to remain aligned.

2.2 Requirement Analysis

The requirements fall into four groups: viewer access and taste capture; movie discovery and playback; recommendation ranking; and administrative or operational support. Viewer preferences are captured through likes and watched-film events. Discovery uses catalogue metadata and visible match indicators. Ranking is requested from the backend rather than re-created as local scores when the service is unavailable. Operational requirements include a PostgreSQL connection, a JWT secret, API-origin configuration, and the recommendation artifact files.

2.3 System Architecture

The application is divided into a browser frontend, a FastAPI service and persistent data/model resources. The frontend uses HTTP requests to authenticate and synchronise preference actions. The API validates the request, resolves the user from a bearer token, reads or updates persistent state, and loads the recommendation data/model artifacts for ranking. The frontend README specifies the API base URL as http://localhost:8000/api by default, configurable with VITE_API_URL.

Table 2.1: High-level system architecture

Layer	Responsibilities	Communicates with
React frontend	Catalog, search, onboarding, profile, movie details and playback	FastAPI endpoints
FastAPI service	Authentication, taste updates, recommendation scoring and stream lookup	PostgreSQL and model artifacts
PostgreSQL	Users, likes and configured movie streams	Backend via SQLAlchemy
Model/data artifacts	Movie metadata, content matrix and fitted SVD estimator	Recommendation endpoint

2.4 Data and Model Preparation

For the content component, the supplied notebook screenshots show a TF–IDF vectorizer configured with English stop words, a genre-text matrix and a cosine similarity computation. The backend README explains that a user taste profile is created from matching liked-movie rows, then compared with the catalogue to obtain content scores. For the collaborative component, the notebook screenshot shows a Surprise Dataset built from user, movie and rating columns, a trainset, and an SVD model with 50 factors and random_state 42. The trained model is saved as a joblib artifact.

The README describes independent min–max normalisation of the content and collaborative scores before blending. Let C_i be the normalised content score for movie i, S_i the normalised collaborative prediction, and w the content weight. The hybrid score is $H_i = wC_i + (1 - w)S_i$. Already liked movies are excluded before the top-N results are selected. The documented endpoint defaults to top_n=6 and weight_a=0.5.

Table 2.2: Recommendation signals and blending

Signal / stage	Description
Content score	Cosine similarity between the liked-movie profile and catalogue TF–IDF vectors.
Collaborative score	SVD-predicted rating for each catalogue movie and current user.
Normalisation	Each score array is scaled separately to the range 0 to 1.
Hybrid score	weight_a × content score + (1 − weight_a) × collaborative score.
Filtering	Exclude already-liked titles, then return the highest-ranked top_n items.

2.5 Authentication and API Integration

The backend README documents signup and login endpoints that return a bearer access token. Protected operations require the token in the Authorization header. The token uses HS256, expires after 24 hours, and the README states a password length of 8–72 characters. The user identity used for protected likes and watch events comes from the verified token rather than the email field in the request body.

The documented routes include POST /api/signup, POST /api/login, POST /api/like, POST /api/watch-event, DELETE /api/likes, GET /api/hybrid-recommendations and GET /api/stream/{movie_id}. A watch event with watched_percentage at or above 0.5 adds the title to the taste profile when it is not already present. A stream route checks for the requested movie and then a configured default stream.

2.6 Feature Implementation

Table 2.3: Major application modules

Module	Documented work
Account access	Email/password sign-up and sign-in, JWT bearer session.
Taste onboarding	Select starter movies to initialise preference signals.
Discovery	Catalog feed, search, genre filtering and sorting by title, rating or year.
Personalised feed	Backend hybrid ranking, match indicators and unavailable state on failure.
Movie details	Title information, related titles, likes and video playback.
Saved profile	Persistent local session and liked-movie state; backend taste synchronization.
Admin workspace	Catalog operations, viewer-profile inspection, weight tuning and audit logs.

2.7 Testing and Deployment Workflow

The supplied frontend README lists the standard local frontend commands and states that the backend must also be configured for sign-in, likes, watch events and personalised ranking. The backend README describes the required PostgreSQL connection and generated artifacts. It also notes that the project has no automated tests in the supplied backend repository and that model artifacts are not present in the provided working tree.

– Install the frontend dependencies and start Vite; start the FastAPI service separately.

– Configure the backend database URL and secret without exposing secrets in source control.

– Verify signup, sign-in, bearer-protected routes, like toggles and watch-event thresholds.

– Check that the recommendation endpoint returns ranked unseen titles for a user with a catalogue-matching like.

– Test the movie stream route and verify frontend error handling when the API is unavailable.

– Before deployment, supply the three trained artifacts, restrict CORS, validate request bounds and confirm production environment variables.

CHAPTER 3: RESULTS AND DISCUSSION / OBSERVATION

3.1 Implemented Application Modules

The project documentation describes a complete movie-discovery workflow spanning accounts, preference capture, search and catalogue browsing, recommendation ranking, movie details and playback. Match percentages and the taste-profile status are visible in the supplied application screenshots. The project README describes admin tools for catalogue management, viewer inspection, recommendation-weight tuning and audit logs.

3.2 User Workflow

Table 3.1: Typical viewer workflow

Step	Viewer action	System response described in README
1	Create an account or sign in	Backend authenticates and returns a bearer token.
2	Choose starter movies	Selected titles seed the recommendation taste profile.
3	Browse or search	Frontend displays catalogue titles and supports filters/sorting.
4	Like a title or watch it	Taste is synchronized; a watch event at ≥50% can add a title.
5	Open recommendations	Frontend requests the backend hybrid ranking endpoint.
6	Open details or play	Details and related titles are shown; stream lookup supplies a URL when configured.

3.3 Recommendation Method

The hybrid design addresses two complementary forms of evidence. Genre-based TF–IDF similarity can identify films whose metadata resembles the titles a viewer likes. SVD collaborative prediction can use rating patterns across users and titles. The blended score makes the relative contribution configurable through the content weight. The frontend documentation specifically states that recommendation scores are not generated locally when the backend recommendation endpoint fails; instead, the UI shows an unavailable state.

This separation makes recommendation results dependent on operational prerequisites: the user needs a valid session and at least one liked title represented in the movie catalogue, and the

backend requires the expected metadata, similarity matrix and trained SVD model files. The missing artifact files are an important reproducibility and deployment constraint.

3.4 Functional Testing and Evidence

The supplied materials contain README descriptions and five screenshots. They do not include test output, automated test results, a live API response or deployment validation. The table below distinguishes reported functionality from verification evidence; it does not mark untested flows as passed.

Table 3.2: Functional checks and evidence status

Check	Expected outcome	Evidence status
Sign-in screen	Email and password form is displayed.	Visible in supplied Figure 6.1; submit flow not independently tested.
Home/feed screen	Featured movie and recommendation status are shown.	Visible in supplied Figure 6.2.
Catalogue browsing	Movie cards, match labels and saved controls appear.	Visible in supplied Figure 6.3.
TF–IDF processing	Genre text is vectorised and cosine similarity is computed.	Notebook code visible in supplied Figure 6.4.
SVD training	Rating dataframe, trainset and fitted model are used.	Notebook code visible in supplied Figure 6.5.
Authenticated API / ranking	Requests return persisted preferences and recommendations.	Documented by README; not independently executed.
Automated regression tests	Test suite runs successfully.	No automated tests supplied in backend README.

3.5 Observations

– The content and collaborative signals complement one another and are combined through an explicit weight.

– Keeping recommendation ranking on the API makes error states clear but requires the backend and model files to be available.

– Using a verified bearer token for user identity is safer than trusting a client-provided email field.

– Frontend local storage supports session continuity but requires deliberate logout and reset behavior.

– Reproducible model data and trained artifacts are essential to operate the recommendation endpoint.

3.6 Limitations

The README says the three required recommendation artifacts are missing from the supplied repository and must be obtained or generated. It also notes that recommendation query values top_n and weight_a do not yet have explicit validation bounds, and that a liked title must match the model catalogue. The documentation describes an in-process likes cache alongside PostgreSQL persistence; multi-instance deployment should rely on shared persistent storage. There are no automated tests or container configuration in the repository described by the README. Performance, recommendation quality, security review and production deployment were not established by the materials supplied for this report.

CHAPTER 4: CONCLUSION

4.1 Conclusion

FlixRecommend is documented as a full-stack movie-discovery project that connects a React frontend to a FastAPI service and combines content-based TF–IDF similarity with collaborative SVD predictions. Its account, preference, catalogue, recommendation and playback workflows provide a practical example of integrating user-facing discovery with a data-driven ranking service.

The supplied screenshots show sign-in, a featured recommendation, catalogue browsing, content-similarity code and collaborative-model training code. The README provides additional details about endpoints, persistence and project setup. The report avoids presenting those materials as independent test results and records the missing model artifacts and unprovided training particulars as items requiring confirmation.

4.2 Future Scope

Future work should provide a repeatable pipeline for building and versioning the movie metadata, similarity matrix and SVD artifact; add automated API and frontend tests; validate recommendation query parameters; and document model data provenance and evaluation metrics. Operational improvements include containerised setup, deployment health checks, stronger production configuration and observability. Recommendation improvements could evaluate ranking quality on held-out interactions and compare alternative content and collaborative weights.

CHAPTER 5: REFERENCES

[1] FlixRecommend project team, “Hybrid Movie Recommendation API: README,” project documentation supplied with this report, 2026.

[2] FlixRecommend project team, “FlixRecommend frontend: README,” project documentation supplied with this report, 2026.

[3] scikit-learn developers, “TfidfVectorizer,” scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[4] scikit-learn developers, “cosine_similarity,” scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html

[5] scikit-surprise developers, “SVD,” Surprise documentation. Available: https://surprise.readthedocs.io/en/stable/matrix_factorization.html

[6] FastAPI, “FastAPI Documentation.” Available: https://fastapi.tiangolo.com/

[7] React, “React Documentation.” Available: https://react.dev/

[8] PostgreSQL Global Development Group, “PostgreSQL Documentation.” Available: https://www.postgresql.org/docs/

[9] SQLAlchemy, “SQLAlchemy Documentation.” Available: https://docs.sqlalchemy.org/

[10] Python Software Foundation, “Python 3 Documentation.” Available: https://docs.python.org/3/

References [3]–[10] are technical documentation sources for the technologies named in the supplied project README; the project-specific implementation descriptions are based on [1], [2] and the attached images.

CHAPTER 6: APPENDIX: ADDITIONAL INFORMATION REGARDING TRAINING

This appendix includes the five screenshots supplied with the request. Captions identify what is visible in each image; they do not imply independent execution or validation of the underlying application or notebook.

Table 6.1: Supplied project screenshots

No.	Image file	Visible subject
1	Screenshot_2026-10-21_122958.png	FlixRecommend sign-in interface.
2	Screenshot_2026-10-21_123428.png	Featured movie, match indicator and backend taste-profile status.
3	Screenshot_2026-10-21_123555.png	Movie catalogue with match indicators and saved controls.
4	Screenshot_2026-10-21_124221.png	Jupyter notebook cells for TF–IDF and cosine similarity.
5	image.png	Jupyter notebook cells preparing rating data and training an SVD model.

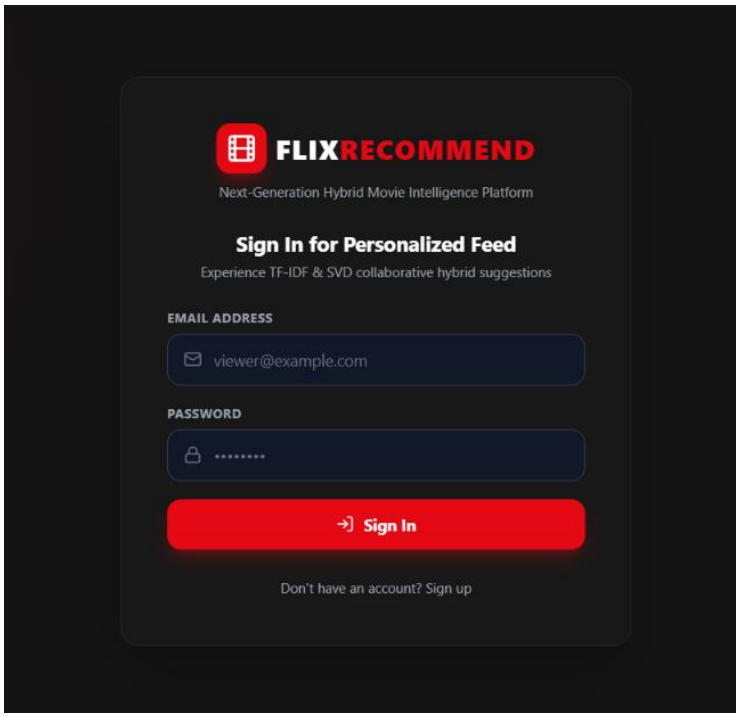


Figure 6.1: Sign-in screen for personalised movie discovery.

The screenshot shows the branded sign-in form with email and password fields. The image is supplied as interface evidence; the successful submission of credentials is not shown.

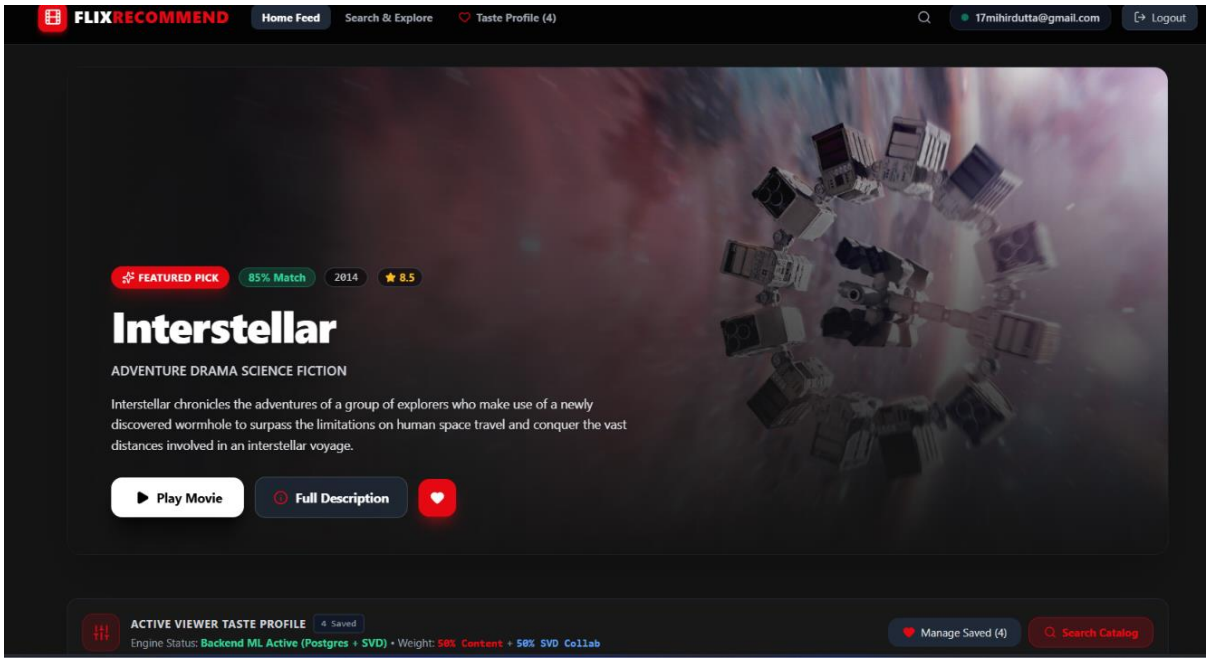


Figure 6.2: Featured movie panel and recommendation status.

The featured-film view displays a match percentage, rating, movie description and play/details controls. Beneath it, the taste-profile panel reports backend ML status and the content/SVD weight split.

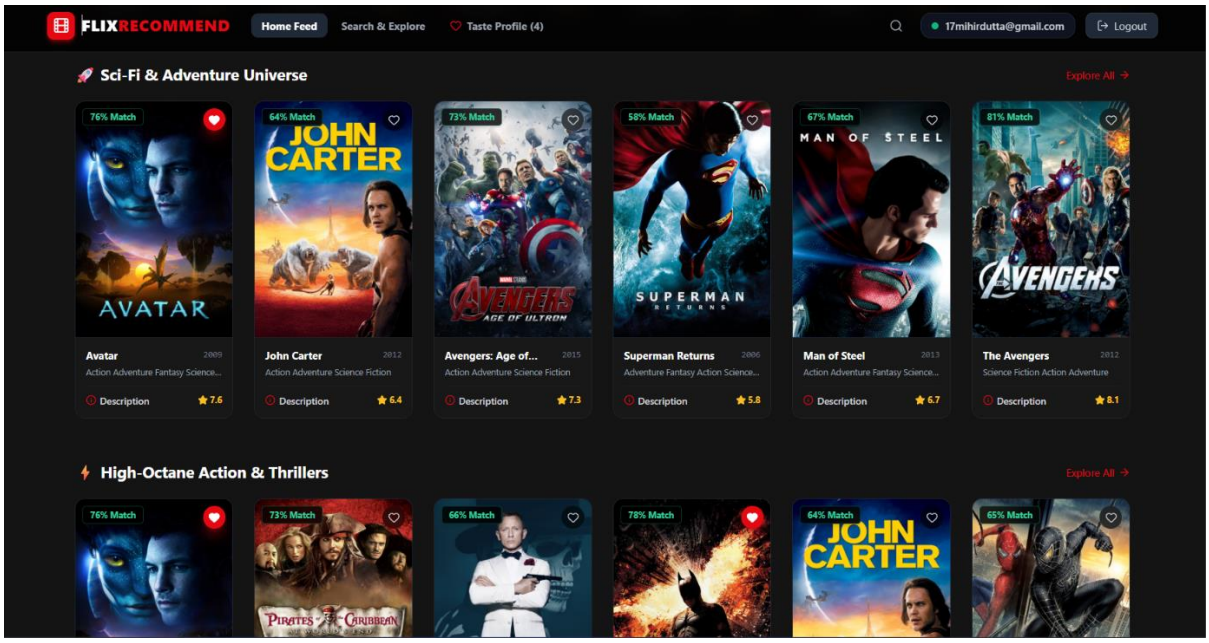


Figure 6.3: Movie catalogue with match indicators and saved controls.

The catalogue screenshot shows movie cards arranged in genre-oriented rows, with match labels, saved controls, release years and ratings.

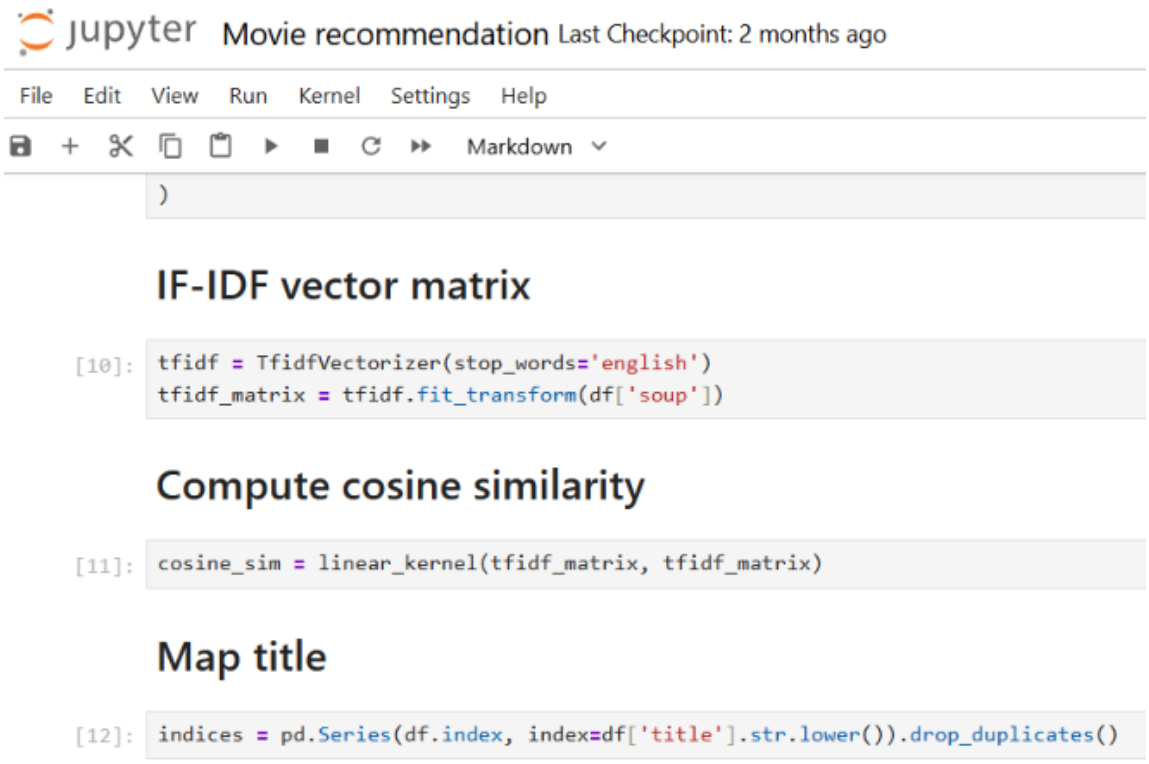


Figure 6.4: Notebook code for TF–IDF vectors and cosine similarity.

The notebook image shows a TF–IDF vectorizer with English stop words, creation of a genre-text matrix and cosine similarity calculation.

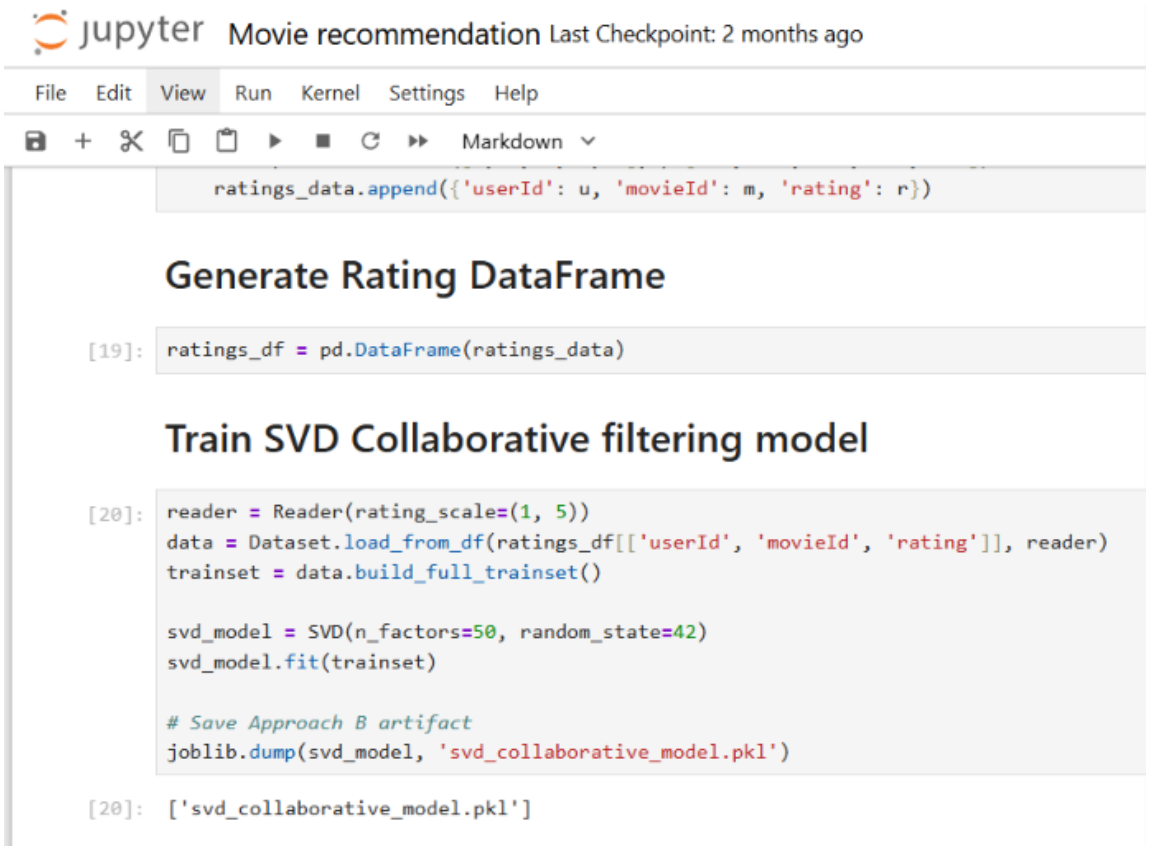


Figure 6.5: Notebook code for rating data and SVD collaborative model training.

The notebook image shows a rating dataframe prepared for a Surprise trainset and an SVD model configured with 50 factors and random_state 42, followed by saving the model artifact.